KnowFeat: Knowledge-Guided Feature Engineering with Auditable Provenance via LLM Agents

Chengsong You[1], Wangyue Li[1], Weiqiao Que[1], Qizhou Chen[1], Wei Deng[2], Feng Zhu[3], Xiaofeng He[1,*]
[1]East China Normal University, Shanghai, China
{51275901122, 52285901017}@stu.ecnu.edu.cn, weiqiaoque@gmail.com, 582476065@qq.com, hexf@cs.ecnu.edu.cn
[2]Southwestern University of Finance and Economics, Chengdu, China
dengwei@swufe.edu.cn
[3]Digital Currency Department, Postal Savings Bank of China, Beijing, China
Jason_bank@126.com

***Abstract*—Automated feature engineering with large language models (LLMs) can generate semantically meaningful features for tabular data, yet existing methods lack structured domain knowledge, rigorous verification, and auditable provenance. We propose KnowFeat, the first knowledge-guided feature engineering framework that addresses all three gaps. KnowFeat organizes domain knowledge into a five-type hierarchy—from schema metadata to regulatory indicators, detection rules, expert opinions, and court document evidence—and injects it as structured context into an LLM agent. A three-stage verification pipeline filters candidates through code execution, statistical quality checks, and model effectiveness evaluation. Every accepted feature carries a provenance card tracing its design to specific knowledge assets, enabling the auditability required in regulated domains. On seven public benchmarks, KnowFeat achieves the best average rank (1.4) among seven methods. On a real-world anti-money laundering dataset with 200K transactions, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over a 93-feature baseline, outperforming methods that generate up to 1,031 unverified features. Ablation studies confirm that hierarchical domain knowledge is essential for consistent feature generation. Code and data are available at https://anonymous.4open.science/r/knowfeat-C967/ and https://huggingface.co/datasets/anonymous-7219/knowfeat.**

I. Introduction

Feature engineering is widely recognized as one of the most impactful steps in building machine learning models for tabular data [1]. Carefully designed features can substantially improve model accuracy, reduce overfitting, and provide interpretable signals for downstream tasks. However, manual feature engineering is labor-intensive, requires deep domain expertise, and does not scale across diverse applications.

To address this challenge, automated feature engineering (AutoFE) methods have been developed. Traditional AutoFE approaches such as OpenFE [2] and AutoFeat [3] apply systematic transformations—mathematical operations, aggregations, and feature crosses—to existing columns, then select the most predictive ones. These methods are purely data-driven: they explore a predefined transformation space without any understanding of what the features *mean* or why certain transformations might be useful for a given domain. In practice, understanding the *semantic* meaning of features is crucial in domains such as finance, healthcare, and marketing, where stakeholders must know *what* a feature represents before deploying it in production.

The emergence of large language models (LLMs) has opened a new paradigm for feature engineering. Methods such as CAAFE [4], FeatLLM [5], and LLM-FE [6] use LLMs to generate feature transformation code based on dataset descriptions. These approaches leverage the LLM's broad world knowledge to propose semantically meaningful features—for example, generating a "debt-to-income ratio" for a credit dataset without being explicitly told to do so. However, current LLM-based methods share three fundamental limitations:

(1) Lack of hierarchical, structured domain knowledge. Most existing methods rely primarily on the LLM's parametric knowledge, which is general-purpose and may be outdated or incomplete for specialized domains. Some recent work incorporates limited forms of domain context: LLM-FE [6] provides dataset descriptions as evolutionary prompts, and RiskTagger [7] annotates Web3 behaviors using regulatory knowledge. However, none of these methods organizes domain knowledge into a *hierarchical, structured* format—from regulatory indicators and detection rules to expert opinions and case evidence—that enables systematic injection and prevents LLM hallucination. In high-stakes applications such as anti-money laundering (AML), such structured knowledge exists in external sources that LLMs cannot access through their training data alone.

(2) Insufficient verification. Existing verification mechanisms vary in rigor but remain incomplete. CAAFE evaluates candidates with a logistic regression proxy that poorly approximates the downstream model. LLM-FE uses downstream model accuracy as feedback but lacks statistical quality filtering. FeatLLM performs no model-based validation. None of these methods combines code execution checking, statistical quality filtering (e.g., information value, redundancy), and full model evaluation into a unified pipeline, leading to the acceptance of noisy, redundant, or harmful features.

(3) Limited auditability. In regulated industries such as finance, healthcare, and legal compliance, model features must be auditable. Decision-makers need to understand *why* a feature was created and *what evidence* supports its relevance.

*Corresponding author.

While some methods provide natural language explanations (e.g., RiskTagger generates annotation rationales), they lack fine-grained, structured provenance that traces each feature back to specific knowledge assets with detailed reasoning chains. Such auditable provenance is essential for deployment in environments that require explainability and regulatory accountability.

In this paper, we propose **KnowFeat** (**Know**ledge-guided **Feat**ure engineering), a framework that addresses all three limitations. Each contribution targets a specific gap identified above:

- **Hierarchical Knowledge Injection** (addressing limitation 1). We organize domain knowledge into five types of increasing specificity—schema metadata, risk indicators, detection rules, expert opinions, and court document evidence—and inject them as structured context into the LLM agent. Each type is independently ablatable, enabling systematic study of knowledge contributions.
- **Three-Stage Verification Pipeline** (addressing limitation 2). Each candidate passes through L1 (code execution sandbox), L2 (statistical quality with noise-calibrated thresholds), and L3 (model effectiveness via XGBoost across 5 seeds), ensuring only genuinely useful features are accepted.
- **Auditable Provenance** (addressing limitation 3). Every accepted feature carries a structured provenance card recording the knowledge assets, reasoning chain, computation steps, and verification results, enabling compliance officers to audit *why* each feature exists.

We evaluate KnowFeat on seven public tabular datasets from OpenML and a real-world AML dataset containing 200,000 digital currency transactions with 93 baseline features. We compare against six baselines spanning traditional AutoFE (OpenFE, AutoFeat), LLM-based (CAAFE, FeatLLM), and evolutionary LLM (LLM-FE) approaches. Key findings include:

- On public datasets, KnowFeat achieves the highest AUC on 4 of 7 datasets and the best average rank (1.4) among all seven methods, while requiring significantly fewer features than alternative LLM-based methods (12–46 vs. 98–1,124 for FeatLLM).
- On the AML dataset, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over a strong 93-feature baseline, substantially outperforming all baselines including LLM-FE (+1.4pp F1).
- Ablation experiments confirm that domain knowledge is essential: the schema-only variant generates zero accepted features on one dataset, and the full knowledge hierarchy outperforms schema-only on 5 of 6 remaining datasets.
- Every generated feature includes a provenance card linking it to specific knowledge assets, providing full auditability.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the KnowFeat framework. Section IV describes our experimental setup and results. Section V illustrates KnowFeat's knowledge-grounded features through case studies. Section VI concludes with limitations and future directions.

II. Related Work

A. Traditional Automated Feature Engineering

Traditional AutoFE methods search a *syntactic* transformation space. Deep Feature Synthesis [8] applies aggregation primitives along relational paths; ExploreKit [9] enumerates operators and ranks candidates via meta-learning; AutoFeat [3] uses polynomial expansion with L1 selection; and OpenFE [2] frames feature generation as an operator-operand search with LightGBM-based importance ranking. End-to-end AutoML systems such as Auto-sklearn [10] and FLAML [11] include feature preprocessing but do not generate semantically new features. These methods apply predefined mathematical operators without *semantic* understanding: `age × income` is treated identically to `age × zip_code`, even though only the former has financial meaning. They cannot generate domain-specific features (e.g., "transaction velocity" in anomaly detection) that require conceptual understanding [12].

B. LLM-Based Feature Engineering

The emergence of large language models [13], [14] has enabled a paradigm shift from syntactic to semantic feature generation. CAAFE [4] iteratively prompts an LLM with dataset descriptions and evaluation feedback to generate Python code for new features, using logistic regression as a proxy evaluator. FeatLLM [5] takes a two-pass approach: first extracting natural language rules from the LLM, then converting these rules into binary indicator features. LLM-FE [6] frames feature engineering as an evolutionary optimization problem, using LLMs to iteratively generate and refine features through structured prompts with model performance feedback. OCTree [15] guides LLM-based feature generation with decision tree reasoning, using tree structure feedback to steer the generation process. In the financial domain, RiskTagger [7] uses an LLM agent with regulatory knowledge to annotate Web3 crypto laundering behaviors, demonstrating that domain knowledge can enhance LLM-based data annotation.

These methods vary in their use of domain knowledge and verification. LLM-FE incorporates limited domain context (dataset descriptions), but none organizes knowledge into a hierarchical, structured format for systematic injection. Verification also varies: CAAFE uses a weak proxy model, LLM-FE uses downstream model accuracy, OCTree uses decision tree feedback, and FeatLLM performs no model-based validation. None combines code execution, statistical quality filtering, and model evaluation into a unified pipeline, nor offers structured provenance that traces each feature to specific knowledge assets.

Tool-augmented LLM agents [16] have been applied to code generation [17], software engineering [18], data analysis [19], and scientific discovery [20]. In data science, AIDE [21] and DS-Agent [22] automate end-to-end pipelines but do not address knowledge-grounded feature generation. KnowFeat

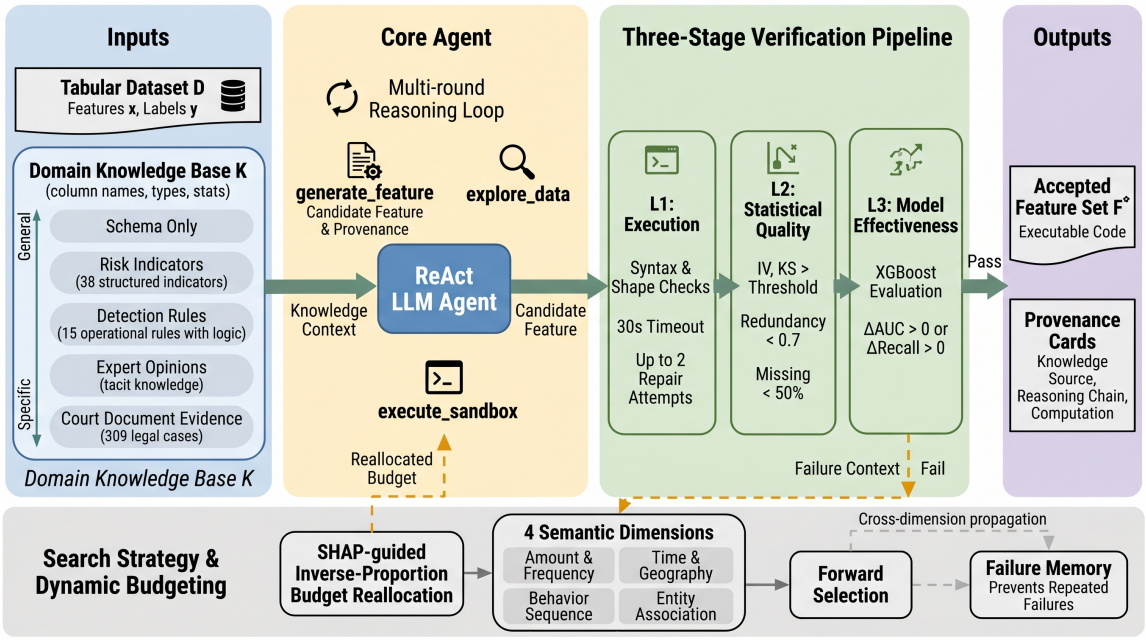


Fig. 1. Overview of the KnowFeat framework. The LLM agent generates features guided by hierarchical domain knowledge. Each candidate is verified through a three-stage pipeline (L1–L3). SHAP feedback dynamically reallocates the exploration budget across semantic dimensions.

builds on the tool-use paradigm by equipping an LLM agent with specialized tools for data exploration, feature generation with provenance, and multi-stage verification.

Table I summarizes the key differences among related methods. KnowFeat is the first to combine hierarchical structured domain knowledge, multi-stage verification (L1–L3), and full feature provenance tracking within an LLM-based feature engineering framework.

C. Domain Knowledge Integration in Machine Learning

Knowledge-infused learning [23] incorporates external knowledge graphs into neural architectures, and transfer learning [24] implicitly transfers knowledge through pretrained representations, but both are designed for unstructured data rather than tabular features. In financial AML, domain experts manually encode risk indicators, structuring patterns, and network centrality measures as hand-crafted features [25]. Regulatory bodies publish red-flag typologies (e.g., FATF guidelines [26]), and institutions maintain internal detection rules from compliance experience and legal proceedings [27]. This rich, structured knowledge has not been systematically leveraged by automated feature engineering methods.

III. Methodology

This section presents the KnowFeat framework in detail. We first provide an overview of the pipeline (Section III-A), then describe each component: hierarchical knowledge injection (Section III-B), the LLM agent with specialized tools (Section III-C), the three-stage verification pipeline (Section III-D), the intelligent search strategy (Section III-E), and auditable provenance output (Section III-F).

TABLE I
Comparison of automated feature engineering methods. ✓: fully supported; ○: partially supported; blank: not supported; –: not applicable.

Property	Open FE	Auto Feat	CAA FE	Feat LLM	LLM- FE	OC Tree	Know Feat
Semantic underst.			✓	✓	✓	✓	✓
Hier. knowledge					○		✓
Exec. verific.	–	–					✓
Statistical filt.	✓	✓					✓
Model validation			○		✓	✓	✓
Provenance track.							✓
Adaptive search					✓	✓	✓

○ Hier. knowl.: LLM-FE=dataset desc. only. Model valid.: CAAFE=LogReg proxy; OCTree=DT feedback.

A. Framework Overview

KnowFeat is a multi-round, knowledge-guided feature engineering framework built on an LLM agent. Given a tabular dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with baseline features $\mathbf{x}_i \in \mathbb{R}^d$ and target label y_i, along with a domain knowledge base $\mathcal{K}$, KnowFeat generates a set of new features $\mathcal{F}^* = \{f_1, f_2, \ldots, f_k\}$ such that the augmented dataset $\mathcal{D}' = \{([\mathbf{x}_i;\ f_1(\mathbf{x}_i);\ \cdots\ ;\ f_k(\mathbf{x}_i)],\ y_i)\}$ achieves improved predictive performance on a target metric (e.g., AUC [28] or Recall).

The overall pipeline operates as follows (see Figure 1 for an illustration):

1) **Initialization.** Load the dataset, compute baseline model performance, and calibrate statistical thresholds via noise injection (Section III-D).

2) **Knowledge Assembly.** Organize domain knowledge $\mathcal{K}$ into a five-type hierarchy and inject the selected types into the agent's context (Section III-B).
3) **Multi-Round Generation.** For each round $r = 1, \ldots, R_{\max}$:
   a) Allocate per-dimension budgets based on SHAP feedback from the previous round.
   b) For each semantic dimension (Section III-E), the LLM agent explores the data, proposes candidate features grounded in knowledge assets, and submits them for verification.
   c) Each candidate passes through the L1→L2→L3 pipeline. Accepted features are added to the baseline; rejected features enter the failure memory.
   d) Forward selection discovers feature combinations that are individually weak but collectively effective.
4) **Convergence.** The process terminates when no new features are accepted for C consecutive rounds (default $C = 2$) or the maximum round count $R_{\max}$ is reached.
5) **Output.** The accepted feature set $\mathcal{F}^*$, along with executable code and auditable provenance cards for each feature (Section III-F).

B. Hierarchical Knowledge Injection

A key innovation of KnowFeat is the systematic integration of external domain knowledge into the feature generation process. We organize knowledge into five hierarchical types of increasing specificity, from general metadata to concrete case evidence:

(i) Schema Metadata. The agent receives only the dataset schema—column names, data types, and basic statistics (mean, cardinality, missing rate). No domain knowledge is provided. This serves as the ablation baseline, testing what the LLM can generate from metadata alone.

(ii) Risk Indicators. Structured risk indicators derived from regulatory guidelines and industry standards. Each indicator specifies a risk dimension, risk class, relevant data field, and directionality. For the AML domain, we compile 38 indicators (IND_01 through IND_38) covering transaction amount thresholds, frequency patterns, temporal anomalies, and network centrality measures. For example, IND_05 flags accounts whose single-day transaction count exceeds a regulatory threshold, while IND_15 identifies abnormal transaction modes such as fan-in and many-to-many patterns.

(iii) Detection Rules. Operational detection rules that encode specific logic patterns. Each rule includes a rule ID, natural language description, logical expression, confidence level, and a Python implementation hint. We compile 15 rules (RULE_001 through RULE_015) from compliance manuals and internal documentation. For example, RULE_006 encodes the "scatter-in, gather-out" layering pattern.

(iv) Expert Opinions. Insights from domain expert interviews that capture tacit knowledge—heuristics, edge cases, and domain intuitions—difficult to formalize as rules but valuable for feature design. We collect 12 expert opinions (EXP_01 through EXP_12). For example, EXP_05 notes that "border region transactions combined with nighttime activity are a hallmark of cross-border laundering."

(v) Court Document Evidence. Risk factors extracted from 309 real court judgments via a key-clue extraction pipeline. This is the most concrete type, grounding feature generation in case evidence (e.g., pyramid-shaped fund network patterns independently validated by legal proceedings).

The five types can be freely combined via a knowledge filter parameter, enabling ablation study of each type's contribution. In the "full" configuration (default), all types are included. Each knowledge item is assigned a unique identifier for provenance tracking.

C. LLM Agent with Specialized Tools

KnowFeat employs an LLM agent (DeepSeek-V3 in our implementation) that interleaves reasoning with tool calls following the ReAct paradigm [16]. The agent is equipped with three specialized tools:

Tool 1: Data Exploration. Executes Python code on the dataset in a sandboxed environment to inspect distributions, compute correlations, and identify patterns before proposing features. The tool returns printed output or return values to the agent.

Tool 2: Feature Registration. Registers a candidate feature with mandatory provenance fields:

- `feature_name`: A descriptive identifier.
- `code`: Executable Python code that computes the feature from raw data.
- `explanation`: A natural language description of what the feature captures.
- `reasoning`: A structured chain linking knowledge assets to the feature design, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]..."
- `knowledge_ids`: A list of specific knowledge asset identifiers (e.g., RULE_001, IND_05, EXP_03) that inspired the feature.
- `computation_steps`: Step-by-step description from raw fields to the final feature value.
- `data_fields_used`: The raw columns consumed by the feature.

Tool 3: Verification Trigger. Triggers the L1→L2→L3 verification pipeline on a registered feature and returns the result (accepted/rejected with detailed reasons). The agent must call Feature Registration followed immediately by Verification Trigger for each candidate—batch submission is prohibited to ensure tight feedback loops.

The agent's prompt instructs it to: (1) reason about which knowledge assets are relevant to the current dimension and budget, (2) explore the data to validate assumptions, (3) propose a feature grounded in specific knowledge items, and (4) submit it for verification and react to the feedback.

D. Three-Stage Verification Pipeline

Each candidate feature passes through three progressively stricter verification stages:

1) L1: Code Execution Sandbox: The generated Python code is executed in an isolated sandbox with a 30-second timeout. L1 checks for:

- Syntax and runtime errors.
- Output shape consistency (one value per sample).
- Presence of required identifier columns.
- Data type validity (numeric output).

If L1 fails, the agent receives the error message and is allowed up to 2 automatic repair attempts before the feature is permanently rejected.

2) L2: Statistical Quality Filtering: The feature's statistical properties are evaluated against automatically calibrated thresholds. To establish these thresholds for discriminative power, we generate $N_{\text{noise}} = 100$ random features (drawn from standard distributions), compute their quality metrics, and set the acceptance threshold at the P_{noise}-th percentile (default $P_{\text{noise}} = 95$). This ensures that accepted features are statistically better than random noise. L2 checks four criteria:

- **Information Value (IV)** [25]: Must exceed the noise-calibrated threshold, ensuring the feature has meaningful discriminative power between positive and negative classes.
- **Kolmogorov–Smirnov (KS) Statistic** [12]: Must exceed the noise-calibrated threshold, measuring the maximum distributional separation between classes.
- **Redundancy**: Maximum absolute Pearson correlation with all existing accepted features must be below 0.7, preventing near-duplicate features from inflating the feature set.
- **Missing Rate**: The proportion of NaN values must be at most 50%, ensuring sufficient coverage for model training.

The noise-calibrated thresholds apply to IV and KS; the redundancy and missing rate thresholds are fixed.

3) L3: Model Effectiveness: The feature is evaluated for its actual contribution to predictive performance. We train an XGBoost [29] model on the baseline features augmented with the candidate feature, using stratified 80/20 train-test splits across 5 random seeds $\{42, 123, 456, 789, 2024\}$. The acceptance criterion depends on the task:

- **Classification (public datasets)**: $\Delta\text{AUC} > 0$, i.e., the augmented model must achieve strictly higher AUC than the baseline.
- **AML detection**: $\Delta\text{Recall} > 0$ *and* $\text{Precision} \geq 90\%$, ensuring that the feature improves detection without sacrificing precision.

The three-stage design is intentionally progressive: L1 is cheap (milliseconds), L2 is moderate (seconds), and L3 is expensive (minutes per feature). By filtering early, we avoid wasting computational resources on clearly defective candidates.

E. Intelligent Search Strategy

To systematically explore the feature space, KnowFeat employs a multi-component search strategy.

1) Dimension-Based Budgeting: We decompose the feature space into D semantic dimensions relevant to the target domain (default $D = 4$ for the AML domain):

1) **Amount & Frequency**: Features related to monetary values and transaction counts.
2) **Time & Geography**: Features capturing temporal patterns and spatial distributions.
3) **Behavior Sequence**: Features encoding sequential patterns in user actions.
4) **Entity Association**: Features derived from relationships between entities (accounts, counterparties).

Each dimension receives an initial budget of B_0 candidates per round (default $B_0 = 3$). After each round, SHAP [30] (SHapley Additive exPlanations) analysis is performed on the current model to estimate each dimension's contribution to predictions.

2) SHAP-Guided Budget Reallocation: Let s_j denote the mean absolute SHAP value for dimension j's accepted features, $B_{\min}$ the minimum per-dimension budget (default 1), and B_{total} the total budget for the round. We reallocate the budget for round $r{+}1$ using an *inverse-proportion* strategy:

$$B_j^{(r+1)} = B_{\min} + (B_{\text{total}} - D \cdot B_{\min}) \cdot \frac{1/s_j}{\sum_{j'=1}^{D} 1/s_{j'}} \quad (1)$$

This ensures that underrepresented dimensions—those contributing least to the current model—receive more exploration budget, promoting balanced feature coverage.

3) Failure Memory: A failure memory $\mathcal{M}_{\text{fail}}$ records all rejected features along with their rejection reasons. Before each generation step, the K most recent failures for the current dimension (default $K = 10$) are injected into the agent's prompt, preventing the regeneration of previously rejected ideas and guiding the agent toward novel directions.

4) Forward Selection: Individual features may fail L3 yet contribute positively when combined with others. We apply greedy forward selection on L2-passed but L3-failed candidates, sorted by L2 quality scores. Each candidate is added to the accepted set if the augmented set improves the target metric, capturing synergistic feature interactions.

5) Cross-Dimension Propagation: After each round, the top-K_{cross} features (default $K_{\text{cross}} = 5$) by SHAP importance are shared across all dimensions as "inspiration prompts," encouraging the agent to adapt successful patterns from one dimension to others.

F. Auditable Provenance Output

A distinguishing contribution of KnowFeat is that every generated feature is accompanied by a structured *provenance card* that enables full auditability. Each provenance card records:

- **Knowledge assets**: The specific identifiers (e.g., RULE_006, IND_15, EXP_05) that inspired the feature design.

TABLE II
DATASET STATISTICS.

Dataset	Samples	Feat.	Pos%	Domain
credit_g	1,000	20	70.0	Finance
diabetes	768	8	34.9	Healthcare
blood_transfusion	748	4	23.8	Healthcare
heart	918	11	55.3	Healthcare
bank_marketing	45,211	16	11.7	Marketing
breast_w	683	9	35.0	Healthcare
adult	48,842	14	23.9	Census
SimECNY (AML)	200,000	93	2.24	Finance

- **Reasoning chain**: A step-by-step explanation of how domain knowledge led to the feature, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]…"
- **Computation steps**: The transformation pipeline from raw data fields to the final feature value.
- **Verification results**: L1/L2/L3 outcomes including IV, KS, redundancy, and ΔAUC/ΔRecall.

This traceability is essential for regulated domains: compliance officers can audit *why* each feature exists, *what evidence* supports it, and *how* it was verified. Section V illustrates provenance cards through concrete examples.

IV. EXPERIMENTS

We evaluate KnowFeat on both public tabular benchmarks and a real-world anti-money laundering (AML) dataset. Our experiments aim to answer four research questions:

- **RQ1**: Does KnowFeat improve predictive performance on public datasets?
- **RQ2**: How does KnowFeat perform on a real-world AML task with severe class imbalance?
- **RQ3**: What is the contribution of domain knowledge?
- **RQ4**: How does KnowFeat balance feature quantity vs. quality?

A. Experimental Setup

1) Datasets: We use eight datasets summarized in Table II: seven public benchmarks from OpenML [28] and one real-world AML dataset.

The seven public datasets are standard tabular classification benchmarks obtained via the OpenML Python API [28]. SimECNY is a real-world digital currency transaction dataset containing 200,000 transactions with 93 baseline features and 4,484 positive (suspicious) samples, representing a realistic AML scenario with severe class imbalance (2.24% positive rate).

2) Baselines: We compare against seven baselines:

- **No-FE**: XGBoost [29] on original features.
- **OpenFE** [2]: Transformation-based AutoFE with LightGBM [31]-based selection.
- **AutoFeat** [3]: Polynomial feature generation with L1 selection.

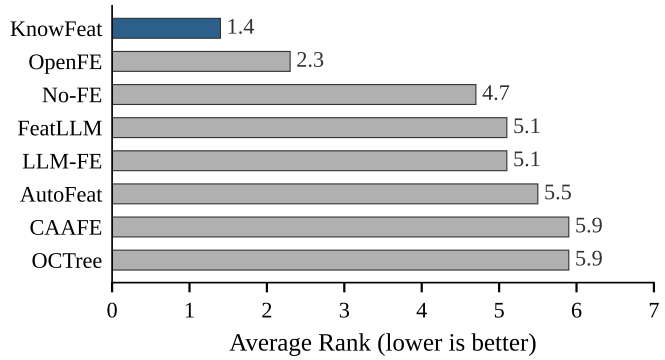


Fig. 2. Average rank across 7 public datasets (lower is better). KnowFeat achieves the best average rank (1.4) among 8 methods.

- **CAAFE** [4]: LLM-based iterative feature generation with logistic regression feedback.
- **FeatLLM** [5]: LLM-based rule extraction and binary feature generation.
- **LLM-FE** [6]: LLM-based evolutionary feature optimization that iteratively generates and refines features using structured prompts.
- **OCTree** [15]: LLM-based feature generation guided by decision tree reasoning.

3) Evaluation Protocol: All methods are evaluated using XGBoost [29] (n_estimators=200, max_depth=6, learning_rate=0.1) with stratified 80/20 train-test splits across 5 random seeds. We use AUC as the primary metric for all datasets, as it is threshold-independent and more robust than accuracy for datasets with varying class distributions. For SimECNY, we additionally report Recall, Precision, and F1 with scale_pos_weight=auto ($\approx$43.6) to handle class imbalance. All baselines are reproduced using our unified evaluation protocol to ensure fair comparison.

B. RQ1: Public Dataset Performance

Table III presents the AUC comparison on seven public datasets. KnowFeat achieves the best performance on 4 out of 7 datasets (credit_g, blood_transfusion, breast_w, and adult) and ranks second on 2 datasets (diabetes and heart).

Figure 2 visualizes the average ranks. KnowFeat achieves the best average rank (1.4), substantially outperforming the second-best OpenFE (2.3).

OpenFE achieves the best results on 3 datasets (diabetes, heart, bank_marketing) through exhaustive operator-operand search. However, OpenFE generates 34–50 features per dataset, while KnowFeat achieves competitive or superior performance with 12–46 features (see Section IV-E).

Among LLM-based methods, LLM-FE generally outperforms CAAFE and FeatLLM but remains below both OpenFE and KnowFeat. OCTree [15], which augments LLM generation with decision tree reasoning, performs comparably to No-FE on most datasets and achieves a near-tied third place on adult, but its average rank (5.9) is among the lowest.

TABLE III
AUC COMPARISON ON PUBLIC DATASETS (MEAN±STD OVER 5 SEEDS). **BOLD**: BEST; UNDERLINE: SECOND BEST.

Dataset	No-FE	OpenFE	AutoFeat	CAAFE	FeatLLM	LLM-FE	OCTree	KnowFeat
credit_g	.767±.021	.783±.031	.754±.017	.760±.023	.760±.022	.762±.026	.763±.066	**.785±.014**
diabetes	.803±.033	**.825±.034**	.809±.027	.803±.030	.810±.032	.815±.033	.803±.034	.819±.032
blood_transfusion	.682±.043	.665±.040	.682±.043	.678±.043	.681±.044	.665±.045	.679±.062	**.701±.036**
heart	.932±.007	**.942±.011**	.934±.003	.930±.006	.930±.004	.933±.005	.930±.022	.938±.006
bank_marketing	.933±.002	**.937±.002**	T/O†	.933±.002	.934±.002	.932±.002	.929±.001	.934±.002
breast_w	.993±.005	.994±.005	.994±.004	.992±.006	.992±.005	.994±.001	.990±.004	**.996±.003**
adult	.929±.004	.930±.003	.925±.004	.929±.003	.929±.003	.928±.003	.930±.002	**.930±.003**
Avg. Rank	4.7	2.3	5.5	5.9	5.1	5.1	5.9	**1.4**
Avg. #Feat.	11.7	41.7	35.1*	14.9	151.3	17.1	12.7	28.4

†T/O: AutoFeat exceeded the 24-hour time limit. *Avg. computed over 6 non-T/O datasets.

TABLE IV
SIMECNY AML RESULTS (SPW=AUTO, 5 SEEDS). n_f: TOTAL FEATURE COUNT INCLUDING 93 BASELINE FEATURES—KNOWFEAT ADDS ONLY 18 FEATURES VS. FEATLLM'S 1,031.

Method	n_f	Recall	Prec.	F1	AUC
No-FE	93	.9663±.010	.9764±.004	.9713±.005	.9996
OpenFE	123	.9645±.008	.9752±.004	.9698±.003	.9996
CAAFE	94	.9681±.008	.9725±.006	.9703±.005	.9996
FeatLLM	1124	.9625±.010	.9728±.005	.9676±.006	.9995
LLM-FE	94	.9699±.007	.9758±.004	.9728±.005	.9996
OCTree	94	.9654±.004	.9756±.003	.9705±.003	.9993
Ours	**111**	**.9871**±.006	**.9855**±.001	**.9863**±.003	**.9999**

Notably, both LLM-FE and OCTree underperform the No-FE baseline on multiple datasets, suggesting that LLM-based generation without external domain knowledge and multi-stage verification is insufficient.

C. RQ2: AML Dataset Performance

Table IV presents results on the SimECNY AML dataset with scale_pos_weight=auto. KnowFeat adds 18 knowledge-guided features to the 93-feature baseline, achieving the best results across all four metrics.

KnowFeat's 18 domain-knowledge features yield substantial improvements: Recall improves by +2.1pp over No-FE (0.9871 vs. 0.9663), F1 by +1.5pp (0.9863 vs. 0.9713), and AUC reaches 0.9999. LLM-FE achieves the second-best F1 (0.9728) with only 1 additional feature, but KnowFeat's margin over LLM-FE remains large (+1.4pp F1, +1.7pp Recall). OCTree adds 1 feature but achieves the lowest AUC (0.9993), while FeatLLM adds 1,031 features yet achieves the *worst* Recall and F1.

To verify robustness, we re-evaluate with scale_pos_weight=1 (no class balancing). KnowFeat remains the clear winner (F1 0.9812) and is the *only* method that outperforms No-FE, confirming that the 18 features encode genuine discriminative signal independent of class reweighting.

TABLE V
ABLATION: SCHEMA_ONLY VS. FULL KNOWFEAT (AUC).

Dataset	schema_only	full	Δ
credit_g	.7750±.012	**.7854**±.014	+.010
blood_trans.	.6891±.048	**.7007**±.036	+.012
heart	.9304±.005	**.9379**±.006	+.008
bank_mkt.	.9314±.002	**.9344**±.002	+.003
adult	.9292±.003	**.9301**±.003	+.001
breast_w	**.9964**±.002	.9957±.003	−.001
diabetes	— (0 feat.)	.8190±.032	—

D. RQ3: Ablation Study

To quantify the contribution of domain knowledge, we compare the *schema_only* variant (schema metadata only) against the *full* variant (all five knowledge types). Results on public datasets are shown in Table V.

On diabetes, schema-only generates *zero* features passing L3, while full knowledge produces 24 features (+0.016 AUC). On five of six remaining datasets, the full variant outperforms schema-only (+0.001 to +0.012 AUC). On blood_transfusion, both variants generate 12 features but differ in quality, confirming that knowledge improves feature *quality*, not just quantity. On SimECNY, the gap is largest: schema-only generates 3 features (+0.3pp Recall) vs. full knowledge with 18 features (+2.1pp Recall), confirming that AML-specific knowledge assets are the primary driver.

E. RQ4: Feature Quantity vs. Quality

The last row of Table III reports the average number of features per method. FeatLLM generates the most features (avg. 151) yet achieves the worst average rank. OCTree and LLM-FE generate few features (avg. 12.7 and 17.1, respectively) but cannot consistently outperform No-FE. On SimECNY (Table IV), KnowFeat's 18 verified features improve F1 by +1.5pp, while FeatLLM's 1,031 unverified features *decrease* F1 by −0.4pp.

KnowFeat strikes an effective balance: it generates a moderate number of features (12–46 on public datasets, +18 on SimECNY) that have all passed the L1→L2→L3 pipeline.

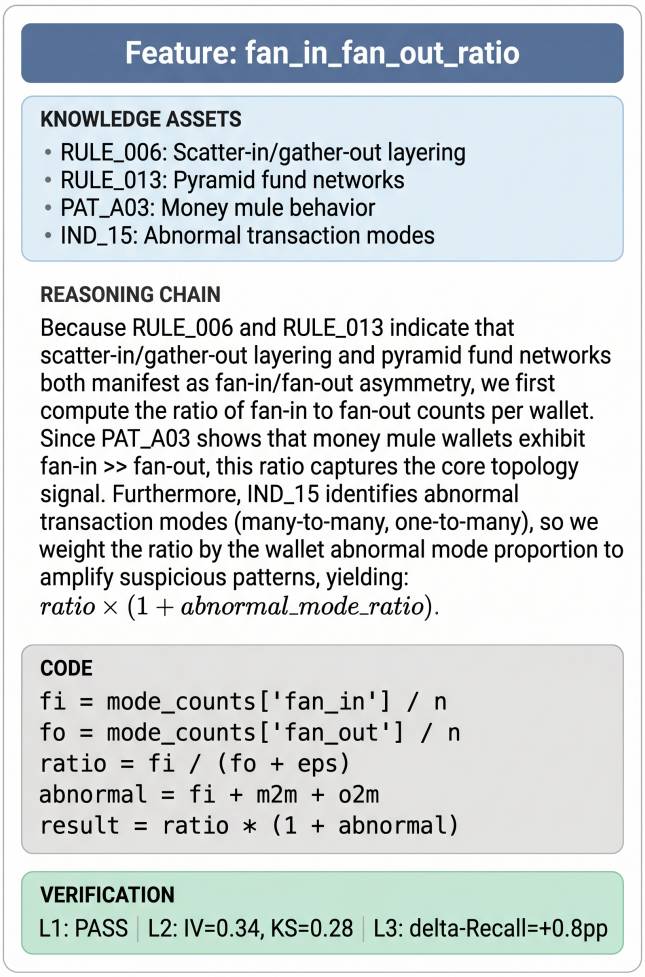


Fig. 3. Provenance card for the `fan_in_fan_out_ratio` feature, showing knowledge assets, reasoning chain, generated code, and verification results.

On SimECNY, the contrast is particularly stark: KnowFeat's 18 verified features improve F1 by +1.5pp, while FeatLLM's 1,031 unverified features *decrease* F1 by −0.4pp. Beyond quantity, every KnowFeat-generated feature includes a provenance card recording the knowledge assets, reasoning chain, and computation steps, providing full auditability (see Section V).

V. Case Study: Knowledge-Grounded Features

We present two representative case studies from the SimECNY AML dataset, highlighting multi-source knowledge fusion and cross-dimensional interaction. Figure 3 shows an example provenance card.

A. Case 1: Fan-In/Fan-Out Ratio (Network Topology)

Feature: `fan_in_fan_out_ratio`—asymmetry between incoming and outgoing transaction patterns, weighted by abnormal mode proportion. Uses 4 knowledge assets: *RULE_006* (scatter-in/gather-out layering), *RULE_013* (pyramid-shaped fund networks), *PAT_A03* (money mule behavioral pattern), and *IND_15* (abnormal transaction modes).

Reasoning chain: (1) Compute fan-in/fan-out mode ratio per wallet from *RULE_006/013*. (2) Weight by abnormal mode proportion from *IND_15*. (3) Combine: ratio × (1 + abnormal_mode_ratio).

```
fi = mode_counts['fan_in'] / n
fo = mode_counts['fan_out'] / n
ratio = fi / (fo + eps)
abnormal = fi + m2m_ratio + o2m_ratio
result = ratio * (1 + abnormal)
```

This feature fuses four knowledge assets spanning three types (rules, patterns, indicators) to produce a topology-aware asymmetry score. No syntactic method would produce this—it requires understanding that directional imbalance combined with abnormal mode prevalence characterizes money mule behavior.

B. Case 2: Cross-Dimensional Risk Interaction

Feature: `high_risk_region_off_hour_ratio`—multiplicative interaction between geographic risk and temporal anomaly. Uses 3 knowledge assets: *RULE_005/014* (cross-regional flows through border areas), *IND_08/09* (off-hour transaction concentration), and *EXP_05* ("border region + nighttime trading is the hallmark of cross-border laundering").

Reasoning chain: (1) Flag high-risk border regions from *RULE_005/014*. (2) Compute off-hour ratio from *IND_08/09*. (3) Neither signal alone is sufficient (*EXP_05*); multiply: is_high_risk × off_hour_ratio.

```
is_hr = region.isin([540000, 650000])
off = (hour >= 18) | (hour < 8)
off_ratio = off.sum() / n
result = is_hr * off_ratio
```

This feature demonstrates *cross-dimensional interaction*: geographic context (account metadata) combined with temporal patterns (transaction logs), guided by expert knowledge. The multiplicative form ensures only the conjunction triggers a high score, exactly as the domain expertise prescribes. Both cases illustrate capabilities that syntactic methods cannot replicate, and both carry complete provenance chains for regulatory audit.

VI. Conclusion

We presented KnowFeat, a knowledge-guided feature engineering framework that combines hierarchical domain knowledge injection, multi-stage verification (L1–L3), and auditable provenance within an LLM agent. On seven public datasets, KnowFeat achieves the best AUC on 4 of 7 and the best average rank (1.4) among eight methods, while requiring significantly fewer features than alternative LLM-based methods. On a real-world AML dataset with 200K transactions, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over a strong 93-feature baseline—gains that are robust across different class balancing settings. The ablation study confirms that domain knowledge is essential, and every generated feature is traceable to specific knowledge assets.

These results have two broader implications. First, the knowledge hierarchy demonstrates that structured external

knowledge can systematically guide LLMs beyond their parametric knowledge, producing features that domain experts recognize as meaningful rather than artifacts of data-driven search. Second, the provenance cards generated by KnowFeat provide a practical pathway toward regulatory compliance for ML-based decision systems: each feature can be audited, challenged, and justified using the same evidence trail that compliance officers already use in manual processes. This framework generalizes beyond AML to any regulated domain where feature interpretability and traceability are prerequisites for deployment.

Limitations and Future Work. Our current knowledge hierarchy is manually organized by domain experts; automating knowledge extraction and structuring from heterogeneous sources (regulations, case law, expert interviews) would reduce the setup cost for new domains. KnowFeat currently uses a single LLM for all reasoning; a multi-agent architecture with specialized agents for knowledge retrieval, feature proposal, and verification could improve efficiency and enable parallel exploration of different semantic dimensions. Additionally, the three-stage verification pipeline is sequential and does not yet support incremental model updates—each L3 evaluation retrains from scratch, which becomes costly as the accepted feature set grows. Exploring online or incremental verification could reduce computational overhead for large-scale deployments.

Extending KnowFeat to additional high-stakes domains such as cybersecurity, insurance fraud detection, and clinical trial analysis would further validate the generality of the knowledge-guided paradigm. We also plan to investigate the integration of retrieval-augmented generation (RAG) techniques for dynamic knowledge retrieval, allowing KnowFeat to access and incorporate evolving regulatory guidelines and emerging threat patterns without manual knowledge base updates.

REFERENCES

[1] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

[2] T. Zhang, Z. A. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, and L. Jian, "OpenFE: Automated feature generation with expert-level performance," in *Proc. of the 40th Int'l Conf. on Machine Learning (ICML)*, 2023, pp. 41 880–41 901.

[3] F. Horn, R. Pack, and M. Rieger, "The autofeat Python library for automated feature engineering and selection," in *Machine Learning and Knowledge Discovery in Databases (ECML-PKDD)*. Springer, 2019, pp. 111–120.

[4] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[5] S. Han, J. Yoon, S. O. Arik, and T. Pfister, "Large language models can automatically engineer features for few-shot tabular learning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.

[6] N. Abhyankar, P. Shojaee, and C. K. Reddy, "LLM-FE: Automated feature engineering for tabular data with LLMs as evolutionary optimizers," *Transactions on Machine Learning Research (TMLR)*, 2025.

[7] H. Li, Y. Xu, F. Zhang, R. Shi, Z. Gu, Y. Yao, Z. Liu, and Z. Li, "RiskTagger: An LLM-based agent for automatic annotation of Web3 crypto laundering behaviors," *arXiv preprint arXiv:2504.17678*, 2025.

[8] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. of the IEEE Int'l Conf. on Data Science and Advanced Analytics (DSAA)*, 2015, pp. 1–10.

[9] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in *Proc. of the IEEE Int'l Conf. on Data Mining (ICDM)*, 2016, pp. 979–984.

[10] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[11] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "FLAML: A fast and lightweight AutoML library," *Proc. of Machine Learning and Systems (MLSys)*, vol. 3, pp. 434–447, 2021.

[12] F. Nargesian, H. Samulowitz, U. Khurana, E. B. Khalil, and D. S. Turaga, "Learning feature engineering for classification," in *Proc. of the 26th Int'l Joint Conf. on Artificial Intelligence (IJCAI)*, 2017, pp. 2529–2535.

[13] T. B. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[14] H. Touvron, T. Lavril, G. Izacard, X. Martinet *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[15] J. Nam, K. Kim, S. Oh, J. Tack, J. Kim, and J. Shin, "Optimized feature generation for tabular data via LLMs with decision tree reasoning," *arXiv preprint arXiv:2406.08527*, 2024.

[16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2023.

[17] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2024.

[19] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu *et al.*, "Data interpreter: An LLM agent for data science," *arXiv preprint arXiv:2402.18679*, 2024.

[20] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[21] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-driven exploration in the space of code," *arXiv preprint arXiv:2502.13138*, 2025.

[22] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.

[23] U. Kursuncu, M. Gaur, and A. Sheth, "Knowledge infused learning (K-IL): Towards deep incorporation of knowledge in deep learning," *Proc. of the AAAI Spring Symposium*, 2020.

[24] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big Data*, vol. 3, no. 1, pp. 1–40, 2016.

[25] Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karuppiah, and K. S. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," *Knowledge and Information Systems*, vol. 57, no. 2, pp. 245–285, 2018.

[26] Financial Action Task Force (FATF), "International standards on combating money laundering and the financing of terrorism & proliferation: The FATF recommendations," Updated 2023, 2012, available at https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html. Accessed: 2025-12-01.

[27] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," *arXiv preprint arXiv:1908.02591*, 2019.

[28] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.

[29] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. of the 22nd ACM SIGKDD Int'l Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[30] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[31] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

APPENDIX

This appendix illustrates the five knowledge types provided to KnowFeat for the SimECNY anti-money laundering dataset ($\sim$200K digital yuan transactions). One representative example per type is shown; similar structured knowledge is compiled for all types (38 indicators, 15 rules, 12 expert opinions, 309 court documents).

(i) Schema Metadata.

Column names, data types, and basic statistics extracted from the transaction data. Example fields: `amount` (numeric), `wallet_type` (categorical, 4 types: anonymous/personal/enterprise/merchant), `hour` (numeric, 0–23), `src`/`dst` (wallet identifiers), `transaction_mode` (8 categories).

(ii) Risk Indicator (from 38-indicator risk taxonomy).

[IND_05] Dimension: transaction behavior anomaly. Risk class: amount & frequency anomaly. Field: single-day transaction count. Direction: accounts whose daily transaction count exceeds regulatory thresholds indicate potential money mule activity for rapid fund channeling.

(iii) Detection Rule (from 15-rule detection rule set).

[RULE_006] *Scatter-in, gather-out.* Category: structuring. Logic: an account's daily incoming transaction count ≥ 20 AND daily total amount $\geq$ 1,000,000, with high incoming but low outgoing transaction count. Python hint: group by `(src, date)`, aggregate count and sum, filter accounts with many small inflows and few large outflows.

(iv) Expert Opinion (from domain expert interviews).

[EXP] Online "running-score" laundering pattern: anonymous wallet $\rightarrow$ real-name wallet $\rightarrow$ bank account chain transfers, typically operating after midnight. Transaction amounts range 300–5,000 yuan, sustained over 10–30 days, with daily volume reaching wallet limits. Key signals: concentrated nighttime transfers, anonymous-to-real-name chains, daily volume at capacity.

(v) Court Document Evidence (from 309 court judgments).

[CASE] Source: criminal judgment on concealing proceeds of crime. Pattern: the defendant used digital yuan wallets across 6 banks (ABC, WeBank, BoCom, PSBC, CIB, CMB) to create a multi-layer fund channeling network. Observable signals: funds transferred via digital yuan APP, routed through multiple bank wallets as intermediaries, corporate-to-personal account conversion before transfer out.

We illustrate the three-stage verification pipeline using the `fan_in_fan_out_ratio` feature from Case Study 1 (Section V).

L1: Code Execution. The generated Python code is executed in a sandboxed environment with a 30-second timeout. The code successfully produces a numeric array of length 200,000 (one value per transaction), with no NaN values and correct identifier columns. Execution time: 0.8 seconds. *Result: PASS.*

L2: Statistical Quality. Four statistical checks are applied:

- **Information Value**: IV = 0.34, exceeding the noise-calibrated 95th percentile threshold of 0.02. This indicates strong discriminative power between suspicious and normal transactions.
- **KS Statistic**: KS = 0.28, exceeding the threshold of 0.05, confirming significant distributional separation between positive and negative classes.
- **Redundancy**: Maximum absolute Pearson correlation with existing features is 0.41, below the 0.7 threshold.
- **Missing Rate**: 0% missing values.

Result: PASS (all four criteria satisfied).

L3: Model Effectiveness. An XGBoost model is trained on the 93 baseline features plus this candidate, using stratified 80/20 splits across 5 seeds. The augmented model achieves Recall = 0.9701 ($\pm$0.008) vs. baseline Recall = 0.9663 ($\pm$0.010), yielding ΔRecall = +0.38pp. Precision remains above 97%, satisfying the $\geq$90% constraint. *Result: PASS—feature accepted into the final set.*

The entire verification process for this feature takes approximately 95 seconds, dominated by L3 model training (88 seconds). Features rejected at L1 or L2 incur minimal cost ($<$2 seconds), which is why the progressive filtering design is critical for computational efficiency.

On SimECNY, KnowFeat generates 52 candidates across 4 rounds: 47 pass L1 (90%), 31 pass L2 (60%), and 18 pass L3 (35%). The progressive filtering avoids 21 unnecessary L3 evaluations, each requiring $\sim$2.5 minutes of XGBoost training across 5 seeds. The 18 accepted features span all four semantic dimensions: 6 in Amount & Frequency, 5 in Time & Geography, 4 in Behavior Sequence, and 3 in Entity Association, reflecting balanced coverage enabled by SHAP-based budget reallocation (Section III-E). Notably, the failure memory mechanism prevents redundant exploration: across 4 rounds, only 3 candidates (5.8%) were rejected for proposing features too similar to previously rejected ones, indicating that the agent effectively learns from prior failures and adapts its strategy in subsequent iterations. The cross-dimension propagation mechanism further contributes to efficiency: in Round 3, the top-performing Amount & Frequency feature inspired 2 successful Time & Geography features by adapting the fan-in/fan-out asymmetry concept to temporal transaction patterns. These results demonstrate that the combination of failure memory, SHAP-guided budgeting, and cross-dimension propagation enables KnowFeat to explore the feature space both efficiently and comprehensively.